# FarmaPredict · Traspaso: limpieza de datos y modelo (NF3)

Nov 5, 2026 · @Alejandro Ronda Navazo

## Contexto

Para NF3 (entrega el 12/11/2026) necesitamos un flujo completo en local: buscar un medicamento, ver si tiene problema de suministro, ver su riesgo a 30 días y ver sustitutos. Este documento traspasa la limpieza de datos y el modelo de línea base a quien los vaya a hacer.

| Parte | Qué entrega | Quién la consume |
|---|---|---|
| Data (ingesta) | Datos raw: snapshots diarios, backfill, presentaciones | Limpieza |
| Limpieza | Ficheros en `data/cleaned/` con las columnas de la tarea 1 y la tabla de incidencias | Modelo y backend |
| Modelo (ML) | `data/cleaned/riesgo_linea_base.csv` (tarea 3) | Backend, endpoint `/medicamentos/{cn}/riesgo` |
| Backend | API FastAPI según `ejemplos_endpoints.txt` | Front |

El backend lee los datos a través de `src/backend/datos.py`. Hoy lee el raw; cuando existan los ficheros de cleaned, solo cambia ese módulo. Por eso los nombres y tipos de columna de este documento son el acuerdo: si alguno cambia, hay que avisar.

Alcance de NF3, alineado con la Fase 1 y con la rúbrica:

- **Modelo:** regresión logística (scikit-learn), que es la *baseline* del diseño de la Fase 1, comparada con una tasa ingenua por principio activo. XGBoost, el modelo principal del diseño, va a NF4.
- **Datos:** ficheros en `data/cleaned/`, servidos al backend a través de `src/backend/datos.py`. PostgreSQL y Docker Compose quedan para NF4; esa capa permite cambiar a PostgreSQL sin tocar la API.
- **Orquestación:** GitHub Actions con cron, que es el plan B que ya contemplaba la Fase 1 frente a Airflow.
- **Rúbrica:** no se evalúa estética ni automatización. 6 de los 10 puntos son de documentación y orden del repositorio, así que el miércoles 11/11 se dedica a eso.

# Fuentes de datos raw

Todo lo raw es el JSON tal cual lo devuelve la API de CIMA, sin transformar.

| Fuente | Qué contiene | Dónde está | Cómo obtenerla |
|---|---|---|---|
| Snapshots de `psuministro` | Incidencias **activas** cada día (~910 registros) | Rama `snapshots`: `raw/cima/psuministro_YYYYmmdd_HHMMSS.json` | `git worktree add ../farmapredict-snapshots snapshots`; se actualiza solo cada día (GitHub Actions) |
| Backfill | Último episodio de suministro de cada CN, activo o resuelto, consultado el 05/11 | Rama `snapshots`: `raw/cima/backfill_psuministro_20261105.jsonl` | Igual que arriba. No se vuelve a generar |
| `presentaciones` | Catálogo de 42.083 CN con `nregistro`, `dcp`, `vtm`, laboratorio, `comerc` | Local, `data/raw/cima/presentaciones_*.json` | `python -m src.snapshot_diario presentaciones` (~2 min) |
| `medicamentos` | Catálogo de 25.462 medicamentos (nregistro) | Local, `data/raw/cima/medicamentos-todos_*.json` | Flujo actual de `src/main.py` |

Formato del backfill (JSONL, una línea por CN):

```
{"cn": "600123", "consultado": "2026-11-05T17:52:10", "respuesta": {"cn": 600123, "tipoProblemaSuministro": 5, "fini": 1778104800000, "ffin": 1780437600000, "activo": false, "observ": "..."}}
```

Si el CN no ha tenido nunca problemas, `respuesta` no trae `fini` ni `activo`, y `observ` vale "Sin problemas de suministro reportados". En la muestra de 200 CN, el 4,5 % tenía episodio.

# Particularidades que hay que respetar

Cada punto de esta lista es un error que ya hemos visto o casi cometido. Saltarse alguno da datos que parecen correctos y no lo son.

1. **Fechas:** `fini`, `ffin` y `estado.aut` son epoch en **milisegundos**, a medianoche de Madrid guardada en UTC. Hay que convertir con `pd.to_datetime(col, unit="ms", utc=True).dt.tz_convert("Europe/Madrid")` antes de quedarse con el día. Sin el `tz_convert`, todas las fechas se adelantan un día (1793746800000 es el 04/11 en Madrid, pero el 03/11 en UTC).
2. **`ffin` en incidencias activas es una previsión** y la AEMPS la cambia: 5 CN la cambiaron entre el 04/11 y el 05/11, y Visipaque tenía `ffin` en noviembre de 2027 y se resolvió el 05/11. Solo es fecha real de fin cuando `activo` es `false`. En las activas, renombrarla a `fecha_fin_prevista`.

3. **CN siempre como texto.** En `/presentaciones` y `/psuministro` viene como string, pero en el backfill `respuesta.cn` viene como **int**. La clave buena es el `cn` de fuera de `respuesta`. Al leer CSV, usar `dtype={"cn": str, "nregistro": str}`.
4. **`nregistro` sin ceros a la izquierda** (`42991`, no `042991`). El CSV antiguo `medicamentos_preparados_api.csv` los tenía rellenados: no mezclarlo.
5. **El backfill solo trae el último episodio de cada CN.** Si un CN tuvo problemas dos veces, el primero no aparece. Hay que tenerlo en cuenta en la etiqueta y explicarlo en la memoria.
6. **El listado `/psuministro` solo trae activas.** El fin de una incidencia se detecta cuando el CN desaparece entre dos snapshots consecutivos.
7. **`estado.aut` puede ser `null`** (presentaciones revocadas, con `estado.rev`).
8. **`tipoProblemaSuministro`:** los códigos 1–9 están documentados (tabla abajo); los 10, 11 y 12 no. Se pueden deducir cruzando con `observ`, que parece ser la descripción literal del tipo.

| Código | Significado | Activas el 05/11 |
|---|---|---|
| 1 | Consultar nota informativa | 6 |
| 2 | Suministro solo a hospitales | 2 |
| 3 | El médico debe valorar otros tratamientos | 53 |
| 4 | Desabastecimiento temporal | 4 |
| 5 | Existen otros con el mismo principio activo y vía | 626 |
| 6 | Existen otros con los mismos principios activos y vía | 69 |
| 7 | Se puede solicitar como medicamento extranjero | 38 |
| 8 | Restringir prescripción a casos sin alternativa | 0 |
| 9 | Distribución controlada por unidades limitadas | 43 |
| 10, 11, 12 | Sin documentar | 46, 14, 11 |

Fuente de la tabla: CIMA REST API – Problemas de suministro (AEMPS, 2019).

# Tarea 1: limpieza

Hay que generar tres ficheros en `data/cleaned/`, siguiendo el patrón de `src/api/limpieza_medicamentos.py`: una función `limpiar_<fuente>(df, ...)` que no lee ni escribe ficheros, y `main.py` como orquestador. Formato: CSV con `;`, como el actual, hasta que el equipo decida si pasa a Parquet.

1. **`presentaciones.csv`**: una fila por CN, a partir del último `presentaciones_*.json`.

| Columna | Origen en el raw | Tipo |
|---|---|---|
| `cn` | `cn` | texto (clave) |
| `nregistro` | `nregistro` | texto |
| `nombre` | `nombre` | texto |
| `laboratorio` | `labtitular` | texto |
| `comercializado` | `comerc` | booleano |
| `dcp_id`, `dcp_nombre` | `dcp.id`, `dcp.nombre` | texto |
| `vtm_id`, `principio_activo` | `vtm.id`, `vtm.nombre` | texto |
| `generico`, `receta`, `huerfano`, `biosimilar`, `ema` | mismos nombres | booleano |
| `fecha_autorizacion`, `fecha_revocacion` | `estado.aut`, `estado.rev` | fecha (Madrid) |
| `fecha_snapshot` | nombre del fichero | fecha |

2. **`psuministro_snapshots.csv`**: todos los snapshots diarios apilados, una fila por `(cn, fecha_snapshot)`.

Columnas: `cn`, `tipo`, `fecha_inicio` (de `fini`), `fecha_fin_prevista` (de `ffin`), `observaciones`, `fecha_snapshot`. Se procesan **todos** los snapshots, no solo el último (hoy `main.py` solo lee el último).

3. **`backfill_psuministro.csv`**: una fila por CN del backfill.

Columnas: `cn`, `tiene_episodio` (si la respuesta trae `fini`), `tipo`, `fecha_inicio`, `fecha_fin` (solo si `activo` es `false`), `activo`, `fecha_consulta`.

Las columnas que usa el backend (`cn`, `nregistro`, `nombre`, `laboratorio`, `comercializado`, `dcp_id`, `dcp_nombre`, `principio_activo`) **no pueden cambiar de nombre** sin avisar.

**Hecho cuando:**

- [ ] Los tres ficheros se generan con un solo comando desde el raw
- [ ] Cero `cn` nulos y cero duplicados por clave
- [ ] Número de filas igual al del raw, o la diferencia explicada en el log
- [ ] Fechas comprobadas a mano con 2–3 casos conocidos (Becloforte `917070`: inicio el 04/11/2026)
- [ ] Cada función `limpiar_*` es pura, para poder testearla con PyTest

## Tarea 2: tabla de incidencias

Es la tabla de la que sale la etiqueta del modelo: una fila por episodio de problema de suministro, uniendo el backfill (pasado, solo el último episodio por CN) y los snapshots (desde el 04/11, todos los episodios). Va en `data/cleaned/incidencias.csv`.

| Columna | Significado |
|---|---|
| `cn` | Código nacional (texto) |
| `fecha_inicio` | Inicio del episodio ( `fini` ) |
| `fecha_fin` | Fin real; vacío si sigue activo |
| `tipo` | `tipoProblemaSuministro` |
| `activa` | Si sigue activo en el último snapshot |
| `origen` | `backfill` , `snapshots` o `ambos` |
| `fin_por_desaparicion` | `true` si `fecha_fin` se dedujo porque el CN desapareció del listado |

Clave: `(cn, fecha_inicio)`. Pasos:

1. **Del backfill:** las filas con `tiene_episodio`. `fecha_fin` = la del backfill si `activo` es `false`; vacío si es `true`.
2. **De los snapshots:** para cada `(cn, fecha_inicio)`, buscar el primer snapshot en el que ya no aparece. Si existe, `fecha_fin` = la fecha de ese snapshot y `fin_por_desaparicion = true`. Si sigue en el último snapshot, está activa.
3. **Unir por `(cn, fecha_inicio)`:** si está en las dos fuentes, `origen = ambos`. Si el backfill trae `fecha_fin` real, gana sobre la deducida.

Si falta algún día de snapshot, la fecha de fin deducida es la del siguiente snapshot disponible. Con dos ejecuciones diarias debería ser raro.

**Hecho cuando:**

- [ ] Una fila por `(cn, fecha_inicio)`, sin duplicados
- [ ] Ninguna fila con `fecha_fin` anterior a `fecha_inicio`
- [ ] Los 8 CN que desaparecieron entre el 04/11 y el 05/11 (por ejemplo `693352`, `989145`, `607593`) salen con `fecha_fin` 2026-11-05
- [ ] El número de incidencias activas coincide con el último snapshot

## Tarea 3: modelo baseline (NF3), regresión logística

Para NF3 hace falta un modelo entrenado y bien evaluado: la regresión logística que la Fase 1 define como *baseline*, comparada con una tasa ingenua. Es la referencia que XGBoost tendrá que superar en NF4. El trabajo pesado es el dataset (fechas de corte y etiqueta); entrenar el modelo encima cuesta poco.

**Etiqueta.** Para una fecha de corte *t*, un CN es positivo si tiene una incidencia con `fecha_inicio` en (*t*, *t* + 30 días]. Los CN que ya tienen una incidencia activa en *t* se excluyen de ese corte: queremos predecir problemas nuevos, no los que ya existen.

**Fechas de corte.** Semanales. La última posible es el 06/10/2026, porque su ventana de 30 días tiene que terminar antes de la fecha del backfill (05/11). La primera depende de lo que muestre el backfill: el sesgo de "solo último episodio" crece hacia atrás, así que no conviene ir más allá de unos meses. Propuesta inicial: desde mayo de 2026.

**Features** (siempre calculadas con datos anteriores a *t*):

- Número de presentaciones comercializadas con el mismo `dcp` y con el mismo `vtm` (cuántas alternativas hay)
- `generico`, `huerfano`, `biosimilar`, `receta`, `ema`
- Antigüedad de la autorización en años
- Incidencias previas del propio CN y del mismo laboratorio antes de *t*

**Modelos:**

1. **Tasa ingenua (referencia):** riesgo = proporción de CN del mismo `vtm` con incidencia en las ventanas de entrenamiento; tasa global si el grupo tiene menos de 5 CN. Sirve para saber si el modelo aporta algo.
2. **Regresión logística** (scikit-learn), con `class_weight="balanced"` por el desbalanceo y las variables numéricas escaladas. Es el modelo que sirve la API.

**Evaluación.** Split temporal: entrenar con los cortes más antiguos y evaluar con las últimas 4 semanas, nunca un split aleatorio. Métrica principal: PR-AUC de los dos modelos, comparada con la prevalencia (la PR-AUC de un modelo aleatorio). Accuracy no sirve: con

~3 % de positivos, decir siempre "no" da un 97 %. Si la regresión logística no supera a la tasa ingenua, se documenta como resultado; la Fase 0 ya lo contemplaba.

**Salida:** **`data/cleaned/riesgo_linea_base.csv`**, una fila por CN del catálogo. El backend la lee tal cual para `/medicamentos/{cn}/riesgo`.

| Columna | Ejemplo |
|---|---|
| `cn` | `917070` |
| `riesgo` | `0.12` (entre 0 y 1) |
| `nivel` | `bajo`, `medio` o `alto` |
| `modelo` | `regresion_logistica_v0` |
| `horizonte_dias` | `30` |
| `fecha_calculo` | `2026-11-08` |

Los umbrales de `nivel` los elige quien haga el modelo y los deja escritos (por ejemplo: `alto` si el riesgo es al menos 3 veces la tasa global).

**Sesgos que hay que documentar en la memoria:** el backfill solo trae el último episodio por CN, así que en cortes antiguos faltan positivos (sobre todo en los CN con problemas recurrentes); y los snapshots completos solo existen desde el 04/11.

**Hecho cuando:**

- [ ] Dataset con fechas de corte, etiqueta y features, generado con un script
- [ ] Tasa ingenua y regresión logística entrenadas con split temporal
- [ ] `riesgo_linea_base.csv` con una fila por CN, con las columnas de arriba, generado con la regresión logística
- [ ] PR-AUC de los dos modelos frente a la prevalencia, anotada en el README o en `docs/`
- [ ] Rango de fechas de corte, umbrales de `nivel` y sesgos documentados

## Calendario y decisiones abiertas

El código se congela el miércoles 11/11 con el tag de entrega; después solo se toca el documento. Lo que no esté para entonces va a "Limitaciones" y "Pendiente para NF4" del PDF.

1. **Viernes 06/11:** revisar este documento y `ejemplos_endpoints.txt`. Primer entregable: `presentaciones.csv`, porque el backend y el recomendador lo necesitan antes que el resto.
2. **Domingo 08/11:** `psuministro_snapshots.csv`, `backfill_psuministro.csv` e `incidencias.csv`.
3. **Lunes 09/11:** `riesgo_linea_base.csv`, para integrarlo con el backend ese mismo día.
4. **Martes 10/11:** tests de las funciones de limpieza, PR-AUC y sesgos documentados.
5. **Miércoles 11/11:** congelación del código y tag.

Decisiones que hay que cerrar entre limpieza, ML y Data antes del domingo:

- [x] ~~PostgreSQL y Docker Compose: se aplazan a NF4 (decidido el 05/11)~~
- [ ] Quién hace la tarea 2 (tabla de incidencias)
- [ ] Qué tipos cuentan como positivo: todos, o excluir los 5 y 6 (hay alternativa con el mismo principio activo, el 76 % de las activas)
- [ ] Primera fecha de corte, cuando se vea el backfill completo por años
- [ ] CSV o Parquet para `data/cleaned/`

## Tablero para el kanban

Una fila por tarjeta, ordenadas por fecha límite. Hay 26 tarjetas de aquí a la entrega; las que pasen del miércoles 11/11 sin terminar van a "Pendiente para NF4" en el PDF.

| Tarjeta | Responsable | Fecha límite | Estado |
|---|---|---|---|
| Capa de datos del backend (`datos.py`) | Alejandro | 05/11 | En curso |
| Backfill completo y subida a la rama `snapshots` | Alejandro | 05/11 | En curso |
| Revisar el contrato de la API y este documento | Todos | 06/11 | Por hacer |
| Recomendador de sustitutos por DCP | Alejandro | 06/11 | Por hacer |
| Limpieza del repo (`.gitignore`, `turtle`, `import *`, borrar `exploracion_psuministro.py`) | Alejandro | 06/11 | En curso |
| Merge de `api` a `main` por PR | Alejandro | 06/11 | Por hacer |

| Tarjeta | Responsable | Fecha límite | Estado |
|---|---|---|---|
| `presentaciones.csv` (tarea 1) | Limpieza (por confirmar) | 06/11 | Por hacer |
| API FastAPI con los 5 endpoints del contrato | Alejandro | 07/11 | Por hacer |
| Front Streamlit con datos falsos según el contrato | Mohamed | 07/11 | Por hacer |
| `psuministro_snapshots.csv` y `backfill_psuministro.csv` (tarea 1) | Limpieza (por confirmar) | 08/11 | Por hacer |
| `incidencias.csv` (tarea 2) | Por confirmar | 08/11 | Por hacer |
| Dataset: fechas de corte, etiqueta y features (tarea 3) | Francisco | 08/11 | Por hacer |
| Pantallas: búsqueda, ficha, sustitutos y riesgo | Mohamed | 08/11 | Por hacer |
| Cerrar las decisiones abiertas | Limpieza, Francisco, Alejandro | 08/11 | Por hacer |
| Tasa ingenua y regresión logística con split temporal (tarea 3) | Francisco | 09/11 | Por hacer |
| `riesgo_linea_base.csv` (tarea 3) | Francisco | 09/11 | Por hacer |
| Conectar el front a la API real | Mohamed | 09/11 | Por hacer |
| Integración del flujo completo | Alejandro y todos | 09/11 | Por hacer |
| Tests PyTest: API, recomendador y funciones `limpiar_*` | Alejandro y Limpieza | 10/11 | Por hacer |
| README: guía de ejecución desde cero, dependencias y variables de entorno | Alejandro | 10/11 | Por hacer |
| PR-AUC, umbrales de nivel y sesgos documentados | Francisco | 10/11 | Por hacer |
| Capturas: front, backend ( `/docs` ) y flujo completo | Mohamed | 10/11 | Por hacer |
| Congelar el código, tag `v0.3-nf3` y release con los datos | Alejandro | 11/11 | Por hacer |

| Tarjeta | Responsable | Fecha límite | Estado |
|---|---|---|---|
| Prueba desde un clon limpio siguiendo solo el README | Alguien que no sea Alejandro | 11/11 | Por hacer |
| PDF: arquitectura real vs diseñada, índice de código, módulos, limitaciones y pendiente para NF4 | Todos | 11/11 | Por hacer |
| Entrega en Blackboard (PDF individual + enlace al repo y al tag) | Todos | 12/11 | Por hacer |

La prueba desde un clon limpio la hace alguien distinto de quien escribe el README: es la única forma de saber si la guía funciona "desde cero", que es lo que puntúa la rúbrica (criterios 2 y 4).